

Parser generators with Pika Parsing and VM as application

Sabawoon Enayat, Oubayy Ahale, Abdellah El Moussaoui

University of Antwerp

Belgium

sabawoon.enayat@student.uantwerpen.be, oubayy.ahale@student.uantwerpen.be,

abdellah.elmoussaoui@student.uantwerpen.be

The project is composed of 2 separate but coupled parts:

1. Parser generator using Pika parsing [1].
2. A virtual machine for a small ASM instruction set.

Pika parsing is a dynamic programming, right to left, bottom-up parsing algorithm that is inspired by Packrat parsing. For any generated parser the runtime performance will be linear as in Packrat. Also error recovery is easier with Pika parsing. We can also think of Pika parsing as a faster and more efficient version of CYK [2], both can check whether a string belongs to a grammar and generate a parse tree.

The parser generator will be used as a library in our project to define a parser using a Parsing Expression Grammar, crafted by the developer of the application, to parse input given by the user and have a parse tree that can be used to do some task systematically. These tasks can include executing instructions in the case of the VM or to use the parse tree to convert a language to a different language (compiling/transpiling), but is not limited to these applications.

The VM will have a 32-bit architecture and as such we choose that the instruction set for the VM will contain 32 instructions, including arithmetic, logical, branch/jump, memory and system operations. The registers and memory will also be visible in verbose mode. Instructions (or code read by the VM to execute) can be either in ASM or a binary format.

The list of the instructions is:

Mnemonic	Type	Description
add	Arithmetic	Addition of two values (from register or immediate)
addu	Arithmetic	Unsigned addition of two values (from register or immediate)
sub	Arithmetic	Subtraction of two values (from register or immediate)
subu	Arithmetic	Unsigned subtraction of two values (from register or immediate)
mul	Arithmetic	Multiplication of two values (from register or immediate)
div	Arithmetic	Division of two values (from register or immediate)
mod	Arithmetic	Modulus of two values (from register or immediate)
inc	Arithmetic	Increment a register
dec	Arithmetic	Decrement a register
and	Logical	Bitwise AND

or	Logical	Bitwise OR
xor	Logical	Bitwise XOR
not	Logical	Bitwise NOT
shl	Logical	Logical left shift
shr	Logical	Logical right shift
sar	Logical	Arithmetic right shift
jmp	Branch/Jump	Unconditional jump to an address
beq	Branch/Jump	Relative branch if equals
bne	Branch/Jump	Relative branch if not equals
bgt	Branch/Jump	Relative branch if is greater
blt	Branch/Jump	Relative branch if is less
call	Branch/Jump	Call a “function”
ret	Branch/Jump	Return from “function”
lw	Memory	Load 32-bit word from memory address into register
lb	Memory	Load byte from memory address into register
sw	Memory	Store 32-bit word from register into memory address
sb	Memory	Store byte from register into memory address
push	Memory	Create space on stack
pop	Memory	Clear the space on stack
mov	Memory	Move data from register into another register
sys	System	Perform system call (Special system operations, like print,...)
nop	System	No operation

Bibliography

[1] <https://arxiv.org/abs/2005.06444>

[2] <https://arxiv.org/pdf/2005.06444> Page 7, Section 2.1: Overview of pika parsing